

项目整体路线图 (SFT + GRPO)

面向团队成员快速掌握 Hull Tactical 竞赛所需的 **SFT (Supervised Fine-Tuning)** 与 **GRPO (Group Relative Policy Optimization)** 流程，聚焦模型侧；量化策略与回测交由专门同学后续接入。

1. 总体 Pipeline 回顾

1. 数据准备

- 基础：官方 `train.csv`、`test.csv`。
- 补充：自建因子、宏观特征、行业映射等，可打包成 Kaggle Dataset。
- 工具栈：`polars`（高效读写）、`pandas`（便利分析）、`numpy`（数值操作）。

2. 特征工程 (Feature Engineering)

- 滑动窗口、曝光归一化、异常值处理。
- 技术指标 (MA、RSI、ATR 等)、宏观周期信号 (PMI、Rate)、横截面排名。
- 建议封装 `FeatureStore`：保持训练、推理一致。

3. 监督微调 SFT

- 目标：预测 `market_forward_excess_returns` 或生成目标仓位，模仿“老师”信号。
- 模型：基于 Attention 的时间序列网络（参见推荐列表）。
- 损失：`MSE`、`Huber`、或带风险惩罚的加权损失。
- 输出：模型权重、Scaler、特征配置。

4. 强化学习 GRPO

- 环境：以滚动窗口回放 `train.csv` / 扩展数据。
- 策略：载入 SFT 权重作为初始 policy。
- 回报： $\text{reward} = \text{pnl} - \lambda \cdot \text{drawdown} - \eta \cdot \text{turnover} - \gamma \cdot \text{risk_penalty}$ （由量化同学定标）。
- 算法：实现或改写 GRPO/PPO 变体，支持连续动作（仓位比例）。

5. 推理部署

- notebook 冷启动加载 SFT + RL 调整后的权重、Scaler、特征映射。
- `predict` 内部维护滑窗缓存，全局变量记录第一次初始化状态。
- 在 Kaggle 环境通过 `DefaultInferenceServer` 进行在线测试。

2. 学习路径建议 (按角色分工)

A. 算法工程同学 (SFT + Attention 模型)

- 先跑官方 **baseline**：重跑 `starter_notebook.py`，确保熟悉 Kaggle 网关。
- 直接上手的优秀架构（附 Quick Start）：
 1. **PatchTST**（短/中期窗口表现稳定）
 - GitHub: <https://github.com/yuqinie98/PatchTST>

- 快速步骤: `git clone`→`pip install -r requirements.txt`→运行 `scripts/PatchTST_stl.sh`; 将 `--data_path` 指向我们预处理后的 CSV。
- 关键文件: `models/PatchTST_backbone.py` (模型结构)、`exp_patchtst.py` (训练入口)。

2. Temporal Fusion Transformer (TFT)

- 代码仓: Google 官方 <https://github.com/google-research/tft>, 同时推荐 Lightning 版 https://github.com/PyTorchLightning/pytorch-lightning-bolts/blob/master/pl_bolts/models/timeseries/temporal_fusion_transformer.py。
- 快速步骤: 直接使用 `lightning-bolts` 提供的 `TemporalFusionTransformer` 类, 配合 `TimeseriesDataSet`; 我们只需适配 `group_id`、`time_idx`、`target` 字段。

3. Autoformer / FEDformer (长序列, 适合月度/季度因子)

- All-in-one 仓库: <https://github.com/zhouhaoyi/Informer2020> (包含 Informer、Autoformer、FEDformer)。
- 快速步骤: 在 `scripts/` 下对 `ETTh1` 任务的命令换成我们数据路径; 注意 `--seq_len`、`--label_len`、`--pred_len` 对齐我们的滑窗长度。

• 无需重写的高阶库:

- `Nixtla / neuralforecast`: 提供 PatchTST、NHITS 等统一接口, 可通过 `pip install neuralforecast` 用 DataFrame 训练。
- `timeseriesAI / tsai`: `Learner` 封装完备, 适合快速试错。

• 工具建议: 使用 PyTorch Lightning 或 Hydra 组织实验配置; 结合 Weights & Biases 做训练监控。

B. 强化学习同学 (GRPO)

• 推荐直接 fork 的实现:

1. Hugging Face TRL: <https://github.com/huggingface/trl>

- 快速入口: `examples/scripts/grpo.py`; 把 `reward_fn` 替换为我们量化奖励函数即可。
- 优点: 与 Transformers 无缝衔接, 支持 GPU/多卡。

2. DeepSpeed-Chat GRPO:

<https://github.com/microsoft/DeepSpeedExamples/tree/master/applications/DeepSpeed-Chat>

- 快速入口: `training/grpo/step3_train_rlhf.py`; 适合较大模型, 内置 `Group Advantage` 计算。

3. OpenRLHF: <https://github.com/OpenLLMAI/OpenRLHF>

- `examples/grpo/grpo_lora.py` 可直接借鉴 reward pipeline, 支持 LoRA 微调 (参数更省)。

• 我们需要改造的点:

- 环境数据: 把每日因子窗口封装成 `Environment`, 输出状态 (特征矩阵) 与奖励 (PnL、风险惩罚)。
- 连续动作: 仓位属于连续区间, 可直接使用 TRL 中的 `ValueHead + tanh` 映射。
- Rule-based reward: 在 `reward_fn` 内计算, 例如 `Sharpe`、`max_drawdown`、`turnover`, 并做标准化/裁剪保持稳定。

• 学习资料 (不需要全文研读, 重点看代码步骤):

- [Group Relative Policy Optimization 论文](#) (关注公式 (3))
- [trl 图文教程: 用自定义 reward 做 GRPO](#)
- [DeepSpeed Blog: Efficient TRL/GRPO Pipeline](#)。

C. 架构/工程同学

- **最小骨架建议:**

1. 使用 `Hydra + Lightning` 构建 `conf/` (配置)、`src/data_module.py` (滑窗 dataloader)、`src/models/` (SFT/GRPO 模块)。
2. 统一 `FeatureStore`: 一个 `features.yaml` 描述因子来源、预处理方式, 训练与推理共用。
3. 模型输出统一管理在 `models/` (SFT 权重 `*.pt`、GRPO Policy `*.pt`、Scaler `scaler.pkl`、特征列顺序 `features.json`)。

- **Kaggle 推理脚本:**

- 在 notebook 顶部加载 `models/` 内容, 初始化 `Predictor`。
- 使用全局缓冲区 `context = deque(maxlen=WINDOW)` 保存最近窗口, 避免重复读取。
- 启动 `DefaultInferenceServer` 前检查加载耗时 (记录 `time.time()` 差值)。

- **工程化资料:**

- [Lightning DataModule 文档](#)
- [Hydra Getting Started](#) (多配置组合)
- [Weights & Biases 快速上手](#) (团队实验跟踪)。

3. 量化奖励与因子挖掘 (GRPO 的关键输入)

“量化部分先不考虑”仅表示当前文档聚焦模型侧, 但在项目中 **reward 设计和因子库是关键**, 需要专人负责并与模型紧密对齐。

1. 奖励函数 (Rule-based RM)

- 参考库:
 - [QuantConnect Alpha Streams 框架](#) (指标定义完善);
 - [FinRL-Meta](#) 中的 `evaluation.py` (提供 PnL/Sharpe/MaxDrawdown 计算)。
- 建议实现:
 - `reward = pnl - λ1 * max_drawdown - λ2 * turnover - λ3 * volatility_penalty;`
 - 将奖励归一化 (例如通过过去 N 堵 reward 均值/标准差) 以稳定 GRPO 更新。
- 需要的输入: 每日预测仓位、实际收益、手续费模型、换手率、风险暴露数据。

2. 因子挖掘与筛选

- 快速工具:
 - [Microsoft Qlib](#): 内置因子库、回测、因子筛选 (IC/ICIR);
 - [ZhangKai-Research / AlphaNet](#) (自动生成技术指标因子);
 - [WorldQuant BRAIN SDK](#) (若许可, 可参考公式生成思路)。
- 建议流程:
 1. 利用 Qlib 的 `F.interday` 功能生成候选因子;
 2. 计算滚动 IC/Rank IC, 筛出 Top-K;
 3. 对通过筛选的因子做去噪/标准化, 写入 `features.yaml`。
- 量化同学产物:
 - `factors.parquet`: 历史因子值表;
 - `reward_config.yaml`: 奖励参数、手续费假设;
 - 回测脚本 (可基于 `backtesting.py` 或自建)。

3. 可借鉴的开源仓库 / 模型架构

方向	推荐仓库	关键特性	适配策略
时间序列 Transformer	PatchTST	Channel-wise patching, 适合金融多资产数据	作为 SFT baseline; 先用滑窗回放训练, 再输出收益预测
长序列预测	Informer / Autoformer	ProbSparse self-attention, 处理长 horizon	可用于多窗口 (30d/60d/120d) 建模, 后续 ensemble
金融量化 Transformer	DeepLOB Transformer	金融 LOB 数据案例, 含数据预处理流程	借鉴其数据标准化与常用回测指标
RLHF / GRPO	Hugging Face TRL	提供 PPO/GRPO 的 PyTorch 实现	改写 reward 函数, 与市场回报挂钩
分布式训练	Colossal-AI RLHF	高效 RLHF pipeline	若模型较大, 可参考其分布式策略
量化回测框架	bt / backtesting.py	快速搭建回测与指标	量化同学用于验证模型输出信号

4. 推荐学习顺序 (两周冲刺版)

时间	内容	输出
第 1 周前半	复现 starter_notebook.py , 掌握 Kaggle gateway; 搭建统一数据加载脚本	数据处理模块、基准模型 (ElasticNet)
第 1 周后半	阅读并复现一个时间序列 Transformer (如 PatchTST), 完成 SFT baseline	Transformer 模型、训练日志、初始权重
第 2 周前半	设计 rule-based reward, 搭建 GRPO 训练脚本; 复用 SFT 权重初始化 policy	GRPO 训练流水线、初版策略权重
第 2 周后半	集成 notebook 推理: 加载 SFT + GRPO 权重, 完成本地 run_local_gateway 测试; 撰写说明文档	推理 notebook、 predict 实现、README 更新

若时间更紧, 可以先上线 SFT 版策略, GRPO 作为增强阶段逐步加入。

5. 关键注意事项

1. 离线训练 → Kaggle 推理

- 训练好的权重、Scaler、因子列表统一上传至 Kaggle Dataset, 命名规范, 如 [models/sft_patchtst_v1.pt](#)。
- Notebook 中使用相同路径读取, 确保在 15 分钟冷启动内完成加载。

2. 模型 ensemble

- 推荐保留多个窗口/模型的预测 (SFT-only、SFT+GRPO、不同时间窗)，在推理阶段做加权或条件选择。
- 注意线上网关实时性，ensemble 步骤需控制在 <5 分钟/批次。

3. 实验追踪

- 记录每次训练的配置 (窗口长度、目标、超参数)。
- 将核心指标 (Sharpe、Max Drawdown、IC、PnL) 写入 `README.md` 或独立日志，方便量化同学对接。

4. 许可证与数据合规

- 任何外部模型/数据在使用前确认 License (MIT/Apache-2.0/GPL 等) 与 Kaggle 规则兼容。
- 对外来资料在团队内部归档，必要时写在提交说明中。

6. 延伸阅读 & 资源清单

- **时间序列 + Transformer**
 - [A Survey on Transformers in Time Series \(2023\)](#)
 - [Temporal Fusion Transformer GitHub](#)
- **GRPO / RLHF**
 - [WeOpenSource/LLM-RLHF 学习笔记](#)
 - [DeepSpeed Chat: Efficient RLHF Training](#)
- **量化与深度学习结合**
 - [AI4Finance-Foundation / FinRL — 深度强化学习量化框架](#)
 - [paperswithcode: Financial Time Series Forecasting](#)
- **Kaggle 实战案例**
 - [Optiver Trading at Scale 竞赛 Top Solutions 概述](#)
 - [Jane Street Market Prediction 方案汇编](#)

7. 下一步执行建议

1. 在仓库新建 `models/`、`configs/`、`features/` 目录，分别存放权重、超参数配置、特征定义。
2. 封装数据流水线，使 `train_loader`、`inference_loader` 共享相同的特征处理函数。
3. 制定 SFT baseline (建议 PatchTST)，形成第一版 notebook 推理脚本。
4. RL 同学基于 Hugging Face TRL 改写 GRPO，优先跑小窗口实验，验证 reward 设计。
5. 每周更新 `README.md` 与 `docs/kaggle_evaluation_notes.md`，同步模型版本与指标表现。

以上内容可直接分配给两位模型同学对照学习，量化同学另行接入奖励函数与回测，实现完整闭环。祝顺利！